Portfolio Rebuild

Implementation Plan v2 — MongoDB · Express · Next.js · Node

Prepared for: Muhammad Waqar | Site: muhammad-waqar.me

This revision locks the stack decision to MongoDB + Express + Next.js + Node — MERN with Next.js replacing plain React as the frontend layer, so the existing MERN skillset carries over directly while SSR/SSG solves the SEO crawlability problem plain React CSR can't. It also incorporates the completed Phase 0 discovery work: the EmailJS audit, the content model, and the two open architecture questions still needing a decision.

Non-Negotiable Constraints (apply to every phase)

1. The contact form / email service must never break. The current EmailJS integration must be re-tested after every structural change, not only at project end.
2. No functionality regressions. Navigation, animations, and all working project/demo links must keep working through every phase.
3. Solo-maintainable. No infrastructure that requires a team to operate.

Plan at a Glance

Item	Detail
Phase 0	Discovery & Architecture Decision — MERN + Next.js stack, custom CMS decision, EmailJS audit, conte
Phase 1	UI/UX Enhancement — unique art direction, fix hero bug, content rewrite, information architecture
Phase 2	SEO — On-Page & Technical — Next.js SSR/SSG, meta/schema, Core Web Vitals, sitemap
Phase 3	CMS Integration — custom Express + MongoDB admin panel for projects, experience, education, skills
Phase 4	SEO — Off-Page & Authority — backlinks, profile cross-linking, indexing, social/OG validation
Phase 5	QA, Performance Audit & Launch — cross-device testing, Lighthouse pass, monitoring setup

Phase 2's SSR/SSG setup must be finalized before Phase 3's CMS wiring, since how the Express API is queried (build-time vs request-time) depends on it. Phase 4 can start once staging is live.

Phase 0 — Discovery & Architecture Decision

This maps directly to Phase 0 of the original plan and incorporates the discovery work already done.

Open Questions — Your Input Needed

The two items below need a decision before the deployment skeleton is created.

1. CMS Approach

Given the decision to build on MERN, the recommendation here shifts from a third-party headless CMS (Sanity/Contentful) to a **custom authenticated admin panel**: Express + MongoDB + Mongoose for the API, JWT-based auth for the admin routes. This keeps the entire stack inside tools already in active use, avoids a recurring third-party CMS bill/limits, and gives full schema ownership. Trade-off: more upfront build time than dropping in Sanity, and you own the admin UI yourself. If build speed matters more than stack purity, Sanity remains a valid fallback — flag if you'd rather go that route.

2. Staging & Hosting

Recommended split: **Vercel** for the Next.js frontend (staging + production, zero-config SSR/ISR support), **Render or Railway** for the Express API (persistent Node process — Vercel serverless functions are a poor fit for a stateful Express app with MongoDB connections), and **MongoDB Atlas** free tier for the database. Confirm you're comfortable with this three-piece split, or name a preferred alternative (e.g. a single VPS running all three).

Framework Migration

- Frontend: Next.js (App Router) with SSR/SSG, replacing the current React/Vite CSR build.
- Backend: Express + Node, connected to MongoDB via Mongoose — the API layer serving both the public site (project/experience/education data) and the admin CMS routes.
- Reasoning: the current CSR build serves an empty HTML shell to crawlers and link-unfurl bots; Next.js SSR/SSG fixes this while Express + MongoDB keeps the CMS/API work inside the existing MERN skillset.

Contact / Email Service Audit

Integration: EmailJS (`@emailjs/browser`)

Current file location: `src/Components/contact/Contact.jsx`

Dependencies (currently hardcoded in source):

- Service ID: `service_95ug8p8`
- Template ID: `template_xl97ao4`
- Public Key: `R9fDbaj9KoVL-LL8k`

Action item: extract these into environment variables during the rebuild — `NEXT_PUBLIC_EMAILJS_SERVICE_ID`, `NEXT_PUBLIC_EMAILJS_TEMPLATE_ID`, `NEXT_PUBLIC_EMAILJS_PUBLIC_KEY` — rather than leaving them hardcoded in source. Note EmailJS's public key is designed to be client-exposed by nature of the library, so the .env move is about config hygiene and easy environment swaps (staging vs production), not secrecy. This satisfies the non-negotiable constraint to protect the existing contact service.

Content Model (MongoDB / Mongoose schemas)

Entity	Fields
Project	`title, shortDescription, problem, roleDecisions, techTags [array], codeLink, liv`
Experience	`role, company, duration, description`
Education	`degree, institution, year`
Skill	`name, category, icon`
SiteMetadata	`title, description, ogImage`

Each will become a Mongoose model with a matching Express CRUD route set (public GET routes for the site, JWT-protected POST/PUT/DELETE routes for the admin panel).

Verification Plan for Phase 0

- Initialize a Next.js boilerplate locally, alongside a minimal Express + MongoDB API skeleton.
- Confirm CMS approach (custom admin vs Sanity fallback) and hosting split before scaffolding further.
- Stand up a staging environment (Vercel + Render/Railway + Atlas, or the confirmed alternative) fully separate from the live production site.
- Verify the EmailJS integration works end-to-end in a standalone Next.js test component, using the .env-based keys, before proceeding to Phase 1.

Phase 1 — UI/UX Enhancement

Goal: Replace the current templated dark-gradient/terminal-hero identity with one distinct, memorable art direction, and fix structural/content issues flagged in the audit.

1a. Visual identity (pick and fully execute ONE direction)

- Editorial/print-inspired: bold display typography as the hero element, asymmetric grid, generous whitespace — no fake terminal widget.
- Warm, high-contrast light palette with one or two deliberate accent colors — a genuine departure from the sea of dark-mode/purple-gradient portfolios.
- Motion-led minimal base: restrained visual design where scroll-triggered transitions and micro-interactions carry the differentiation.
- Whichever is chosen, the first viewport must create a genuine 'wow' moment within 3 seconds — not more visual noise layered onto the current template.

1b. Structural fixes

- Fix the hero rendering bug (overlapping/ghost text behind name and title on load) — audit z-index and layout-shift causes.
- Reorder the page so Featured Projects appears immediately after the hero/intro, not after long About/Education sections.
- Cut or radically compress the 'Goals & Visions' section — replace vague claims with concrete, evidence-backed content or remove it.
- Move tutorial/clone projects (Gmail Clone, Instagram Clone, etc.) into a clearly secondary, visually de-emphasized 'practice' area separate from real project work.
- Verify WCAG AA contrast across all text/background combinations — check, don't eyeball.

1c. Content rewrite

- Rewrite every featured project description in problem → my role/decisions → outcome format (2–4 sentences), replacing adjective-heavy marketing copy.
- Include one concrete technical detail per project (a challenge solved, a metric, a specific feature).
- Never present an undemoable project as 'Featured' — use the status field (live/in-progress/archived) from the Phase 0 content model to route unfinished work to a secondary section automatically.

Deliverable: Fully restyled, content-corrected site on staging, matching one coherent art direction with no leftover template patterns.

Phase 2 — SEO: On-Page & Technical

Goal: Make every page genuinely crawlable and fast, not just visually presentable.

Technical foundation

- Complete the Next.js SSG/SSR migration from Phase 0 — every route must return meaningful HTML on first response (verify with curl/view-source, not just DevTools).
- Decide per-page rendering strategy: SSG for mostly-static pages (About, Education), ISR for the projects list (revalidates when the Express/MongoDB CMS content changes), SSR only where genuinely needed.
- Semantic HTML throughout: correct heading hierarchy, one h1 per page, proper nav/main/section usage instead of generic divs.

On-page elements

- Dynamic, per-page title and meta description generated from the MongoDB-backed content.
- Open Graph and Twitter Card tags with a real OG image per page (test with actual link-unfurl tools, not just markup review).
- JSON-LD structured data: Person schema on the homepage; CreativeWork/SoftwareSourceCode schema per project.
- Auto-generated sitemap.xml and correct robots.txt.
- Canonical URLs on every route; eliminate duplicate-content paths.

Performance

- Image optimization via next/image (WebP/AVIF, explicit width/height to prevent layout shift), descriptive alt text on every image.
- Core Web Vitals targets: LCP under 2.5s, CLS under 0.1, INP under 200ms — measured and reported via Lighthouse/PageSpeed Insights, not assumed.

Deliverable: Lighthouse/PageSpeed report (mobile + desktop) showing passing Core Web Vitals, plus verified crawlable HTML on every route.

Phase 3 — CMS Integration (Custom MERN Admin)

Goal: Enable content updates (new projects, experience, education, skills) without a code deploy, using a self-built Express + MongoDB admin.

Backend (Express + MongoDB)

- Build Mongoose models matching the Phase 0 content model (Project, Experience, Education, Skill, SiteMetadata).
- Public GET routes for the Next.js frontend to fetch content at build/request time.
- JWT-protected POST/PUT/DELETE routes restricted to a single admin user (you) for content management.
- Image uploads: store via a service like Cloudinary or S3-compatible storage rather than the Express server's filesystem, so images survive redeploys.

Admin panel (frontend)

- A simple authenticated /admin area (can live inside the same Next.js app, gated by a login form calling the Express JWT auth route) with CRUD forms for each entity.
- Enforce the status field (live/in-progress/archived) in the form UI so the Phase 1 rule — never feature an undemoable project — is applied automatically.
- Revalidation: trigger Next.js ISR revalidation (via a webhook/API call from the Express admin routes) whenever content is saved, so edits go live without a manual rebuild.

Security

- Hash the admin password (bcrypt), rate-limit the login route, and keep the JWT secret in environment variables — never hardcoded.
- CORS locked to the known frontend origin(s) only.

Deliverable: Working custom admin where a new project or experience entry can be added and appears live within the revalidation window, with zero code changes required.

Phase 4 — SEO: Off-Page & Authority

Goal: Build discoverability and authority signals outside the site itself, once the rebuilt site is live on staging/production.

Tasks

- Submit sitemap to Google Search Console and Bing Webmaster Tools; confirm indexing status of every page.
- Cross-link the portfolio from GitHub profile README, LinkedIn featured section, Upwork/other freelance profiles, and any existing dev-community profiles (Dev.to, Hashnode, etc.).
- Validate social share previews on actual platforms (LinkedIn, Twitter/X, WhatsApp) to confirm OG tags render correctly in the wild, not just in markup.

- If applicable, publish 1–2 technical write-ups (a project case study or a technique used in a portfolio project) on Dev.to/Hashnode with a canonical or backlink to the portfolio project page — genuine backlink building, not link farms.
- Ensure NAP-style consistency (name, role, links) is identical across LinkedIn, GitHub, portfolio, and resume, since recruiters cross-reference these.

Deliverable: Indexed sitemap confirmation, verified social preview rendering, and at least two authoritative external backlinks/cross-links pointing to the portfolio.

Phase 5 — QA, Performance Audit & Launch

Goal: Confirm nothing is broken, everything is measured, and the site is production-ready before pointing the live domain at it.

Tasks

- Full regression pass: every nav link, project demo/code link, and animation across desktop, tablet, and mobile breakpoints.
- Re-test the EmailJS contact form end-to-end (the non-negotiable constraint from page 1) as the final gate before launch, not just after each phase.
- Run a final Lighthouse/PageSpeed audit on the production build and record the scores for reference.
- Cross-browser check (Chrome, Firefox, Safari, mobile Safari) for layout and animation consistency.
- Set up basic uptime/error monitoring (e.g. UptimeRobot for the Express API, Vercel's built-in analytics for the frontend) so future issues surface quickly.
- Point the live domain at the new deployment and re-verify SEO items (sitemap, meta tags, structured data) resolve correctly on the production URL, not just staging.

Deliverable: Signed-off production launch with regression checklist completed, final performance report, and monitoring in place.

End of plan. Each phase's deliverable should be explicitly confirmed before moving to the next phase, particularly the EmailJS re-test in Phase 5, which is the final safeguard for the constraint stated on page 1. Open questions on page 3 (CMS approach, staging/hosting split) should be answered before Phase 0 is marked complete.